**Лабораторно упражнение 3**

**Обектно ориентирано моделиране. Свързаност на отвоворностите и взаимосвързаност**

## I. Теоретична част

## 1. Свързаност на отговорностите (cohesion) и взаимнасвързаност (coupling)

Въпреки, че тези две понятия са известно още със структурния дизайн, те са неразривно свързани и с обектно ориентирания и затова на тях се отделя специално внимание.

### 1.1. Свързаност на отговорностите (cohesion)

Понятието Свързаност на отговорностите или Кохезия (cohesion) показва до каква степен различните задачи и отговорности в дадена програма или компонент са свързани помежду си т.е. колко фокусирана е програмата (или компонетът) към решаването на една единствена цел. Разделя се на:

- Силна свързаност (strong cohesion)
- Слаба свързаност (weak cohesion)

При силната свързаност на отговорностите, задачите на отделните програмни единици (метод, клас, компонент, подпрограма) са свръзани помежду си и се стремят да решат общ проблем. Силната свързаност е характеристика на качествен програмен код.

#### 2.1.1. Силна свързаност на отговорностите (strong cohesion)

Когато говорим за силна свързаност на отговорностите в даден клас, това означава, че класът описва само един субект. Силната свързаност означава, че класът решава една задача, а не много неща едноверенно. Клас, който прави много неща едновременно е труден за разбиране и поддръжка. Например, ако даден клас едновремено решава математическа задача, печата документ, организира списък, такъв клас има слаба свързаност на отговорностите. По-добре е този клас да бъде разделен на няколко, всеки от които решава само една задача.

Като пример за клас, който има силна свързаност на отговорностите е клас **Math**. Той има една задача – да предостави математически изчисления и константи. Могат да бъдат посочени методите **Sqrt, Sin, Cos, Pow**, както и константите **PI** и **E**.

Силна свързаност на отговорностите при даден метод има, когато методът изпълнява само една задача. Такъв метод е добре написан. Метод, който прави много неща има слаба кохезия и трябва да се раздели на няколко по-прости метода.

Индикатор за това дали има силна или слаба кохезия е изборът на име на клас или метод. Ако програмистът се затруднява да даде име на клас или на метод, въз основа на това какво изпълняват, това означава, че класът (или методът) има слаба свързаност на отговрностите.

#### 2.1.2. Слаба свързаност на отговорностите (weak cohesion)

Слаба свързаност на отговорностите се наблюдава при методи, които вършат няколко задачи едновременно. Понякога се налага тези методи да приемат различни групи параметри, за да извършат различните задачи. Понякога може да се наложи логически несвързани задачи да се обединят в такива методи, но използвнето на слаба кохезия е вредно и трябва да се избягва.

Примери за слаба свързаност на отговорностите са следните:

Пример 1: Слаба свързаност на отговорностите при клас

```
class Point
    {
        private int x, y;
        private int Vx, Vy;
        public void Input()
        {
            Console.Write("Enter a value for x:");
            x = Int32.Parse(Console.ReadLine());
            Console.Write("Enter a value for y:");
            y = Int32.Parse(Console.ReadLine());
        }

        public void Translation()
        {
            Console.Write("Enter a value for Vx:");
            Vx = Int32.Parse(Console.ReadLine());
            Console.Write("Enter a value for Vy:");
            Vy = Int32.Parse(Console.ReadLine());
            x += Vx;
            y += Vy;
            Console.WriteLine("The point has been translated to {0}, {1}", x, y);
        }
}
```

Пример 2: Слаба свързаност на отговрностите за метод

```
class Circle
{
        private int radius;
        public void Area()
        {
            double s= radius * radius * System.Math.PI;
            Console.WriteLine("The area is {0}", s);
         }
}
```

Качественият програмен код притежава силна свързаност на отговрностите. Този код е по-ясен и разбираем, по-лесно се поддържа и презиползва. Обратното, при слаба свързаност на отговорностите всяка една промяна може да засегне някаква друга функционалност. Например, ако една логическа задача се разпростира върху няколко модула, промяната ѝ е много по-трудна. Ако даден компонент върши няколко несвързани задачи, преизползването му също е трудно, защото за да се използва налице трябва да са същите условия, а това може и да не е възможно.

### 2.2. Взаимозависимост (coupling)

Взаимозависимостта, наречена още взаимообвързаност определя до каква степен компонетите (класовете) зависят един от друг. Нивото на взаимозависимостта се дели на:

- Слаба взаимозависимост (loose coupling), която се определя още като „добра самостоятелност“
- Силна взаимозависимост (tight coupling)

Слабата взаимозависимост обикновено се съчетава със силна свързаност на отговорностите и затова обикновено термините „strong cohesion“ и „loose coupling“ вървят заедно.

Слабата взаимозависимост (loose coupling) се характеризира с това, че единиците код (подпрограма, функция, клас, компонент) общуват с другите програмни единици минимално, чрез ясно дефинирани интерфейси. По този начин промяната в един компонент не се отразява на другите. Една такава програмна единица лесно може да бъде презиползвана в друг програмен код, защото е слабо свързано с останалите т.е. със заобикалящата го среда.

Интерфейсът на даден клас трябва да е максимално упростен и да дефинира единствено работата на този компонент като избягва всички ненужни детайли.

Силна взаимосвързаност между два или няколко компонента има при много входни данни и параметри и при неописани в интерфейса характеристики. При използване на много контролни променливи се получава сложна за разбиране логика и в крайна сметка – tight coupling.

Силна взаимосвързаност между два компонента означава, че те не могат да работят независимо един от друг и промяната на единия, води до промяна и на другия. Това води до труден за четене на код, който има големи проблеми с поддръжката. При такъв код не е възможно да се премести даден компонент от него в друг програмен проект.

## II. Задачи за изпълнение

1. Разгледайте програмният код в примерите. Предоложете вариант за решение, с който да се избегне слабата свързаност на отговорностите.
2. Да се състави приложение, с което се организира работа с клас полигон, описан с определен набор от точки. Върху полигона да може да се извършат следните графични трансформации: преместване (транслация) и завъртане (ротация). Приложението да може да пресмята площ и обиколка на полигон. Необходимо е да се моделират класовете, които изграждат приложението.

Упътване:

- *Полигонът се описва с определен брой точки. Необходимо е да се предвиди клас, описващ точка.*
- *Графичната трансформация на полигон е графичната трансформация на всяка една от точките*
- *Транслацията на точка е графична трансформация, при която точка с координати (x,y) се преемства в позиция (x+Vx, y+Vy). Стойностите Vx,Vy определят дължината на вектора на транслация по осите x и y, съответно.*
- *Ротацията на точка е графична трансформация, при която точката се завърта под определен ъгъл спрямо началото на координатната система. Новите координати на точката (x1, y1) се пресмятат по формулата:*

  $x1 = x.Cos\ \alpha – y.Sin\ \alpha$

  $y1 = x.Sin\ \alpha + y.Cos\ \alpha$

  *като α е ъгълът на завъртане.*